

Smart Log

සම්පූර්ණ Code Flow සහ Database පැහැදිලි කිරීම

Tech Titans · Rajarata University of Sri Lanka

Dart lines 19,145 · Tests 327 · SQLite tables 8

මේ ලේඛනයේ නියෝග හැම code කැබැල්ලක්ම ඇත්ත ගොනු වලින් කෙළින්ම උපුටාගත් ඒවා — අනිත් ලියපු ඒවා නොවේ. ඒ නිසා ලේඛනයයි code එකයි අතර වෙනසක් ඇති වෙන්න බෑ.

1. App එක පණ ගැන්වෙන තැන — main.dart

App එක විවෘත කරද්දී පිළිවෙළින් සිදු වන දේ. මේ පිළිවෙළ වැදගත්: settings මුලින්ම load වෙන්නේ නැත්නම් පළමු screen එක වැරදි ක්‍රමයකින් volume එකක් පෙන්වන්න පුළුවන්.

lib/main.dart

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );

  // Load measurement settings before the first frame so no screen ever
  // renders a volume using the wrong method, and route settings changes into
  // the sync queue. A settings failure must never block app start.
  try {
    await UserPreferencesService.instance.load();
    UserPreferencesService.instance.syncCallback =
      (_) => CloudSyncEngine.instance.queueSettings();
  } catch (_) {}

  // Anything left in the queue from a previous run goes now: the app may
  // have been killed mid-upload, or closed in a yard with no signal.
  unawaited(CloudSyncEngine.instance.drain());

  // Watches for sign-in, so a fresh install restores its data and everything
  // already on this phone gets swept up to the cloud once.
  CloudSyncCoordinator.instance.start();

  // Signs out a session left open on a phone put down in a yard. Wired here
  // rather than in the auth screens so there is exactly one place that
  // decides what "idle" means.
  SessionTimeoutService.instance.onTimeout = () async {
    SessionTimeoutService.instance.stop();
    await GoogleAuthService.instance.signOut();
  };

  runApp(const SessionActivityDetector(child: SmartLogApp()));
}
```

පේළියෙන් පේළිය:

පේළිය	කරන දේ	ඇයි?
Firestore.initializeApp	Firestore එක ආරම්භ කරනවා	මේක නැතුව FirebaseAuth.instance කැබෙනවා
UserPreferencesService.load()	Settings load කරනවා	පළමු frame එකට කලින්. නැත්නම් වැරදි volume method එකකින් පෙන්නවා
syncCallback	Settings වෙනස් වුණාම queue එකට	Settings ත් cloud එකට යනවා
CloudSyncEngine.drain()	පරණ queue එක යවනවා	App එක කලින් මරලා තියෙන්න පුළුවන් — upload එක මැද්දේ
CloudSyncCoordinator.start()	Sign-in බලාගෙන ඉන්නවා	Restore සහ backfill ට
SessionTimeoutService.onTimeout	Logout callback එක	'නිකම් තියෙනවා' කියන්නේ මොකක්ද කියලා තීරණය කරන එකම තැන

SessionActivityDetector එකෙන් මුළු app එකම wrap කරලා තියෙනවා. ඒක **Listener** එකක් මිසක් **GestureDetector** එකක් නොවේ — ඒ නිසා යටින් තියෙන button එකකට යන tap එකක් ඒක ගිලලා දාන්නේ නෑ.

2. Database — සම්පූර්ණ පැහැදිලි කිරීම

2.1 Database එක විවෘත වන ආකාරය

lib/database/local_db.dart

```
static Future<Database> get database async {
  if (_database != null) {
    return _database!;
  }

  _database = await _initDatabase();
  return _database!;
}
```

Lazy — database එක ඕන වෙන පළමු මොහොත වන තුරු විවෘත වෙන්නේ නෑ. ඒ නිසා අලුතෙන් install කරපු phone එකක, screen එකකට යන තුරු database ගොනුවක්ම හැඳෙන්නේ නෑ.

2.2 Version constant එක

Magic number එකක් නොවේ

```
/// The migration version this build of the code expects.
///
/// Named rather than repeated, so the version passed to [openDatabase] and
/// the version anything else checks against cannot drift apart -- which is
/// precisely the bug a schema-version display exists to catch.
static const int schemaVersion = 7;
```

ඇයි constant එකක්? `openDatabase` එකට යවන version එකයි, developer screen එකේ compare කරන version එකයි එකම එකක් වෙන්න ඕන. දෙකක් වෙන වෙනම ලිව්වොත් ඒවා අතර වෙනසක් ඇති වෙන්න පුළුවන් — සහ ඒක තමයි schema version display එකක් තියෙන්නේම හොයාගන්න.

2.3 Tables — CREATE TABLE කේතය

stacks — ලෝග් කට්ටලයක්

onCreate

```
CREATE TABLE stacks(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT,
  totalVolume REAL,
  totalCost REAL DEFAULT 0,
  createdAt TEXT,
  customerName TEXT,
  remarks TEXT,
  cloudId TEXT
);
```

logs — එක ලෝග් එකක්

```
onCreate

CREATE TABLE logs(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  stackId INTEGER,
  diameter REAL,
  lengthFeet REAL,
  volume REAL,
  cost REAL DEFAULT 0,
  createdAt TEXT,
  measurementSource TEXT,
  rawDiameterInches REAL,
  deductionInches REAL,
  diameterToleranceInches REAL,
  measurementQuality TEXT,
  diameterProfile TEXT,
  cloudId TEXT
)
''');
```

measurementSource, rawDiameterInches, deductionInches, diameterToleranceInches, measurementQuality, diameterProfile — මේ columns 6 තමයි provenance (මූලාශ්‍රය). මාස 6කට පස්සේ ගැනුම්කරුවෙක් volume එකක් ගැන තර්ක කළොත් මේවා තමයි උත්තරය.

Column	තේරුම
measurementSource	sensor එකෙන්ද, keyboard එකෙන්ද?
rawDiameterInches	deduction එකට කලින් තිබ්බ ඇත්ත මිනුම
deductionInches	bark allowance එක කීයද අඩු කළේ?
diameterToleranceInches	මිනුමේ අවිනිශ්චිතතාව
measurementQuality	scan එක කොච්චර හොඳද?
diameterProfile	අක්ෂය දිගේ හැම cross-section එකකම විෂ්කම්භය

defects — දෝෂ

```
_v6Tables

CREATE TABLE defects(
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  logId INTEGER NOT NULL,
  kind TEXT NOT NULL,
  confidence REAL NOT NULL DEFAULT 1,
  automatic INTEGER NOT NULL DEFAULT 0,
  centreX REAL,
  centreY REAL,
  radius REAL,
  imagePath TEXT,
  overlayPath TEXT,
  createdAt TEXT NOT NULL,
  cloudId TEXT,
  FOREIGN KEY (logId) REFERENCES logs (id) ON DELETE CASCADE
)
''',
```

confidence සහ **automatic** දෙකම නියෝගයන් ඇයි? 'මේ දෝෂය model එකෙන් හොයාගත්තද, මිනිසෙක් mark කළද' සහ 'කොච්චර විශ්වාසද' කියන දෙක. Cutting engine එක 0.60ට වඩා confidence නියෝග දෝෂ විතරයි no-go zone විදිහට ගන්නේ.

cutting_patterns — කැපුම් සැලැස්ම

_v6Tables

```
CREATE TABLE cutting_patterns(  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  logId INTEGER NOT NULL UNIQUE,  
  strategy TEXT NOT NULL,  
  boardWidthMm REAL,  
  boardThicknessMm REAL NOT NULL,  
  bladeThicknessMm REAL NOT NULL,  
  boardCount INTEGER NOT NULL,  
  yieldPercent REAL NOT NULL,  
  boardVolumeCubicFeet REAL NOT NULL,  
  wasteVolumeCubicFeet REAL NOT NULL,  
  sawPasses INTEGER NOT NULL,  
  pricePerCubicFoot REAL NOT NULL DEFAULT 0,  
  createdAt TEXT NOT NULL,  
  cloudId TEXT,  
  FOREIGN KEY (logId) REFERENCES logs (id) ON DELETE CASCADE  
)  
""",
```

logId INTEGER NOT NULL UNIQUE — මේක තමයි වැදගත්ම කොටස. එක ලොග් එකකට එක pattern එකයි. නැවත ගණනය කළොත් පරණ එක replace වෙනවා. මේක **schema** එකෙන් enforce වෙනවා, code එකේ 'පරණ එක delete කරන්න මතක තියාගන්න' කියන එකෙන් නොවේ.

sync_outbox — cloud එකට යන්න බලා ඉන්න වැඩ

_createOutboxTable

```
CREATE TABLE sync_outbox(  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  entity TEXT NOT NULL,  
  operation TEXT NOT NULL,  
  localId INTEGER,  
  parentLocalId INTEGER,  
  cloudId TEXT,  
  parentCloudId TEXT,  
  queuedAt TEXT NOT NULL,  
  attempts INTEGER NOT NULL DEFAULT 0,  
  lastError TEXT  
)  
""",
```

Column	තේරුම
entity	stack / log / standaloneLog / settings
operation	upsert හෝ delete
localId	SQLite row id එක (upsert වලට)
cloudId	Firestore document id (delete වලට — row එක නැති නිසා)
attempts	කී වතාවක් try කළාද
lastError	අන්තිමට වුණු වැරද්ද — developer screen එකේ පෙන්වා

2.4 Migrations — පරණ phone එකක data නැති නොවී

Database එක වෙනස් වුණාම, දැනටමත් phone එකේ තියෙන data නැති නොවී අලුත් columns එකතු වෙන්න ඕන. ඒක තමයි migration එකක්:

onUpgrade

```
onUpgrade: (db, oldVersion, newVersion) async {
  if (oldVersion < 2) {
    await db.execute(
      "ALTER TABLE stacks ADD COLUMN totalCost REAL DEFAULT 0",
    );
    await db.execute(
      "ALTER TABLE stacks ADD COLUMN createdAt TEXT",
    );
    await db.execute(
      "ALTER TABLE logs ADD COLUMN cost REAL DEFAULT 0",
    );
    await db.execute(
      "ALTER TABLE logs ADD COLUMN createdAt TEXT",
    );
  }
}
```

`if (oldVersion < N)` කියන pattern එකෙන් — v2 එකේ ඉන්න phone එකක් v7ට යද්දී, v3, v4, v5, v6, v7 හැම එකක්ම පිළිවෙළින් run වෙනවා.

v5 — cloudId ඇයි?

අනුවාද 5

```
if (oldVersion < 5) {
  // The identity a row has in the cloud, stored next to the row.
  //
  // Without it, restoring onto a new phone gives every row a fresh
  // local id, and the next upload invents a *second* cloud document
  // for data already up there -- the backup would duplicate itself a
  // little more with every reinstall.
  await db.execute("ALTER TABLE stacks ADD COLUMN cloudId TEXT");
  await db.execute("ALTER TABLE logs ADD COLUMN cloudId TEXT");

  await db.execute(_createOutboxTable);
}
```

v7 — SDS schema එකට align කිරීම

_v7Columns

```
static const List<String> _v7Columns = [  
  // SDS: Log(..., notes). EC04 FR12 is explicit that the user "enters  
  // optional notes" before saving, and there was nowhere to put them --  
  // only a stack could carry a remark, not the individual log it was  
  // about.  
  "ALTER TABLE logs ADD COLUMN notes TEXT",  
  
  // SDS: Log(..., diameter_1, diameter_2). Both ends of the log, which is  
  // what a taper-aware formula needs. The scan already derives a full  
  // sectional profile; these record the two ends explicitly so a reading  
  // can be checked against a tape at either end.  
  "ALTER TABLE logs ADD COLUMN smallEndDiameter REAL",  
  "ALTER TABLE logs ADD COLUMN largeEndDiameter REAL",  
  
  // SDS: Stack(..., user_id FK). EC04 FR21 requires that "a user can  
  // retrieve only the records owned by that user" -- which nothing local  
  // enforced, so two accounts on one phone saw each other's stacks.  
  "ALTER TABLE stacks ADD COLUMN userId TEXT",  
  
  // SDS: Stack(..., rate). The agreed price per cubic foot for the batch,  
  // which is not recoverable from a total once logs are added or removed.  
  "ALTER TABLE stacks ADD COLUMN rate REAL",  
  
  // SDS: Defect(..., severity ENUM('Low','Medium','High')). Kept as TEXT  
  // because SQLite has no ENUM; the allowed values live in the Dart enum  
  // that writes it.  
  "ALTER TABLE defects ADD COLUMN severity TEXT",  
];
```

2.5 Outbox එකේ dedup — වැදගත් trick එකක්

enqueueUpsert

```
static Future<void> enqueueUpsert({  
  required String entity,  
  required int localId,  
  int? parentLocalId,  
}) async {
```

මොකද වෙන්තේ: එකම stack එකට upsert එකක් දැනටමත් queue එකේ තියෙනවා නම්, පරණ එක delete කරලා අලුත් එකක් දානවා.

Upsert එකේ **payload** එකක් නෑ — යථා වෙලාවේ row එක නැවත කියවනවා. ඒ නිසා stack එකකට වෙනස්කම් 20ක් කළත් upload එකක් විතරයි. සති දෙකක් offline වැඩ කළත් queue එක අසීමිතව වැඩෙන්නේ නෑ.

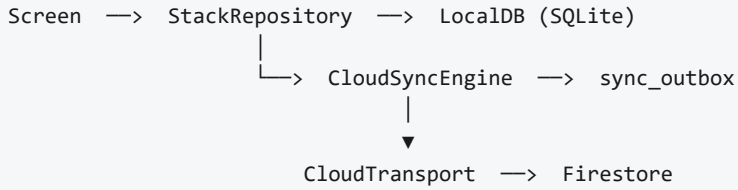
2.6 Search — SQL එකේම filter කරනවා

searchStacks

```
static Future<List<Map<String, dynamic>>> searchStacks({  
  String? keyword,  
  DateTime? from,  
  DateTime? to,  
}) async {
```

ඇයි Dart එකේ නොකර SQL එකේ? Entries 500ක් නියෙද්දී retrieval තත්පර 2ට අඩු වෙන්න ඕන (NFR). හැම row එකක්ම memory එකට අරගෙන බොහෝ ඒවා විසි කරොත්, user එකාට history එකතු වෙන ගමන්ම හෙමින් වෙනවා.

3. Data ගලා යන මාර්ගය (Data Flow)



නීතිය: Screens කවදාවත් LocalDB එකට කෙළින්ම කතා කරන්නේ නෑ. ඒවා යන්නේ StackRepository එක හරහා. ඒ නිසා 'save' කරාම sync queue එකටත් දාන්න ඕන' කියන එක එකම තැනක තියෙනවා.

3.1 StackRepository — save කරන ආකාරය

stack_repository.dart

```
Future<int> addLogToStack({
  required int stackId,
  required double diameter,
  required double lengthFeet,
  required double volume,
  double cost = 0,
  LogMeasurement? measurement,
  double? deductionInches,
}) async {
```

`unawaited(_syncStack(stackId))` — sync එක **await** කරන්නේ නෑ. User එකා log එක save කරාම වහාම ඊළඟ එකට යන්න පුළුවන්. Cloud එකට යන එක පස්සෙන් වෙනවා.

3.2 Delete කරද්දී පරිස්සම් වෙන්න ඕන තැනක්

deleteStack

```
Future<void> deleteStack(int id) async {
  // Read the cloud identity before the row goes: afterwards there is
  // nothing left to look it up from, and a delete that cannot name its
  // document leaves the copy in the cloud behind for ever.
  final stackRow = await LocalDB.getStack(id);
  final logRows = await LocalDB.getLogsForStack(id);

  await LocalDB.deleteStack(id);

  // Removing the stack in the cloud takes its logs subcollection with it,
  // so the logs need no delete entries of their own. What they must not
  // keep is a queued *upsert*: draining that after the stack is gone would
  // recreate the subcollection and leave orphaned logs behind for ever.
  for (final log in logRows) {
    await LocalDB.dropQueuedUpsert(SyncEntity.log, log["id"] as int);
  }
}
```

මෙතන **bug** එකක් තිබුණා. Stack එකක් delete කරාම, ඒක ඇතුළේ තිබිලා logs වලට queue එකේ තිබිලා **upsert** ඒවා ඉතුරු වුණා. ඒවා පස්සේ drain වුණාම, delete කරපු stack එකේ subcollection එක නැවත හැදෙනවා — orphan logs cloud එකේ සිටිනවා. `dropQueuedUpsert` එකෙන් ඒක නවතිනවා.

4. Cloud Sync — ගැඹුරින්

4.1 Queue එකට දාන එක

cloud_sync_engine.dart

```
Future<void> queueLog(int logId, {int? stackId}) async {
  await LocalDB.enqueueUpsert(
    entity: stackId == null ? SyncEntity.standaloneLog : SyncEntity.log,
    localId: logId,
    parentLocalId: stackId,
  );
  unawaited(drain());
}
```

4.2 Drain — queue එක හිස් කිරීම

drain()

```
Future<void> drain() {
  final running = _inFlight;

  if (running != null) {
    // Work arrived mid-flight: go round again rather than making it wait
    // for the next trigger, and hand this caller the same finish line.
    _drainAgain = true;
    return running;
  }
}
```

මෙතන තිබුණු bug එක: කලින් drain() එකක් ද්‍රවනවා නම්, දෙවෙනි call එක වහාම return කළා. ඒ කියන්නේ await drain() කියන එක බොරුවක් — upload එක තාම යනකොට 'ඉවරයි' කිව්වා. දැන් in-flight future එකම return කරනවා.

4.3 Fail වුණම මොකද වෙන්නේ

`_drainOnce()`

```
for (final item in items) {
    try {
        await _send(uid, item);
        await LocalDB.removeSyncItem(item["id"] as int);
    } catch (error) {
        await LocalDB.recordSyncAttempt(item["id"] as int, "$error");
        CloudSyncStatus.instance.recordFailure(
            error,
            operation: "backing up a ${item["entity"]}",
        );

        // Leave the rest queued and come back to it.
        _scheduleRetry();
        await _refreshPending();
        return;
    }
}

CloudSyncStatus.instance.recordSuccess();
_retryTimer?.cancel();
await _refreshPending();
}
```

පළමු fail එකේදීම නවතිනවා — ඉස්සරහට යන්නේ නෑ. ඇයි? Queue එක පිළිවෙලට තියෙන්නේ. Stack එකක් cloud එකේ නැතුව ඒක ඇතුළේ log එකක් ලියන්න බෑ. Fail එකක් පනිනවා නම් cloud එකේ orphans විසිරෙනවා.

4.4 cloudId — duplicate නොවීමේ රහස

`_cloudIdFor()`

```
Future<String> _cloudIdFor(
    String table,
    int localId,
    Object? existing,
) async {
    final current = existing as String?;
    if (current != null && current.isNotEmpty) return current;

    final deviceId = await LocalStorageService.getOrCreateDeviceId();
    final minted = "${deviceId}_${localId}";

    await LocalDB.setCloudId(table, localId, minted);

    return minted;
}
```

Device id එකත් එකතු කරන්නේ ඇයි? Phone දෙකක් offline log එකක් add කළොත් දෙකටම local id 7 ලැබෙන්න පුළුවන්. Device id එකෙන් cloud document id දෙක වෙනස් වෙනවා.

4.5 Transport — ඇයි interface එකක්?

cloud_transport.dart

```
abstract class CloudTransport {
  Future<void> upsertStack(
    String uid,
    String cloudId,
    Map<String, Object?> data,
  );

  Future<void> upsertLog(
    String uid,
    String stackCloudId,
    String cloudId,
    Map<String, Object?> data,
```

Engine එකේ වැඩේ — පිළිවෙළ, retry, කිසිවක් නැති නොවීම — ඒක තමයි නිවැරදිව තියෙන්න ඕන කොටස. නමුත් ඒක real Firestore එකකට එරෙහිව device එකක් නැති machine එකක test කරන්න බෑ. Interface එකක් නිසා test වලදී **fake cloud** එකක් දාන්න පුළුවන්:

test/cloud_sync_test.dart

```
class FakeCloud implements CloudTransport {
  final Map<String, Map<String, Object?>> stacks = {};
  final Map<String, Map<String, Map<String, Object?>>> logs = {};
  final Map<String, Map<String, Object?>> standalone = {};
  Map<String, Object?> settings = {};

  /// When set, every call throws it. Stands in for no signal, or for rules
  /// that reject the write.
  Object? failure;

  int writes = 0;

  void _check() {
    if (failure != null) throw failure!;
```

4.6 Restore — අලුත් phone එකකට

cloud_restore_service.dart

```
Future<RestoreOutcome> restore() async {
  final uid = _uidReader();
  if (uid == null) return const RestoreOutcome();

  final snapshot = await _transport.downloadAll(uid);

  var restoredLogs = 0;

  for (final stack in snapshot.stacks) {
    final cloudId = stack["cloudId"] as String?;

    final localStackId = await LocalDB.insertRestoredStack(stack);

    // Logs are written under the *new* local id their parent just got,
    // while keeping the cloud id they arrived with. Getting this wrong is
    // how a restore silently reparents someone's timber into the wrong
    // customer's stack.
    for (final log in snapshot.logsByStack[cloudId] ?? const []) {
      await LocalDB.insertRestoredLog(log, stackId: localStackId);
      restoredLogs++;
    }
  }

  for (final log in snapshot.standaloneLogs) {
```

Logs අලුත් local stack id එකට යන්න ඕන. මේක වැරද්දනොත් කෙනෙකුගේ දැව වැරදි ගැනුම්කරුවෙකුගේ stack එකට යනවා.

5. ඇල්ගොරිතම — ඇත්ත කේතය සමඟ

5.1 Ellipse Fitting

ලෝග් මුහුණත ඇලයෙන් ඡායාරූප ගත්තොත් ඒක ellipse එකක්. Rays 72ක් යවනවා, නමුත් සමහරක් shadow එකකට වදිනවා. Ellipse එකක් fit කිරීමෙන් ඒ වැරදි ඒවා **outvote** වෙනවා.

Ellipse.radiusAt()

```
double radiusAt(double angle) {
    final local = angle - rotation;
    final c = semiMinor * math.cos(local);
    final s = semiMajor * math.sin(local);

    final denominator = math.sqrt(c * c + s * s);
    if (denominator <= 0) return 0;

    return semiMajor * semiMinor / denominator;
}
```

වැදගත්ම තේරුම් ගැනීම

```
/// When a round face is photographed at an angle, the minor axis is
/// foreshortened but the major axis still spans the true diameter. So this
/// -- not the mean of the two -- is the figure to scale from.
double get trueDiameter => semiMajor * 2;
```

5.2 Log Face Mask — O(1) fit check

log_face_mask.dart

```
bool containsRect(Rect rect) {
    if (rect.width <= 0 || rect.height <= 0) return false;
    if (!rect.left.isFinite || !rect.top.isFinite) return false;
    if (!rect.right.isFinite || !rect.bottom.isFinite) return false;

    final c0 = ((rect.left - origin.dx) / cellSize).floor();
    final r0 = ((rect.top - origin.dy) / cellSize).floor();
    final c1 = ((rect.right - origin.dx) / cellSize).floor();
    final r1 = ((rect.bottom - origin.dy) / cellSize).floor();

    if (c0 < 0 || r0 < 0 || c1 >= cols || r1 >= rows) return false;
    if (c1 < c0 || r1 < r0) return false;

    final cellCount = (c1 - c0 + 1) * (r1 - r0 + 1);

    return _sum(c0, r0, c1, r1) == cellCount;
```

Summed-area table එකක් නිසා ඕනෑම හතරැස්ක එකම ගණනයකින් check කරන්න පුළුවන් — හතරැස් ලොකු කම කුමක් වුණත්.

5.3 Sawing Engine — search එක

planCant()

```
static SawPlan? planCant(SawingRequest request) {
  if (!request.isValid) return null;

  SawPlan? best;

  for (var i = 0; i < _angleSteps; i++) {
    final angle = math.pi * i / _angleSteps;

    final context = _RotatedFace.build(request, angle);
    if (context == null) continue;

    for (final swapped in _orientations(request)) {
      final plan = _packCant(request, context, swapped);
      if (plan == null) continue;

      if (best == null ||
          plan.boardVolumeCubicFeet > best.boardVolumeCubicFeet) {
        best = plan;
      }
    }
  }

  return best;
}
```

Angles $12 \times$ orientations 2 = candidates 24ක්. හැම එකකම volume එක ගණනය කරලා හොඳම එක තෝරනවා.

ඇයි අර්ධ භ්‍රමණයක්?

```
/// Pattern rotations tried, spread over a half turn.
///
/// A half turn rather than a quarter because a log face is not
/// symmetric -- the flat side left by a previous cut, or the dent where a
/// branch came off, makes 10 degrees and 100 degrees genuinely different.
static const int _angleSteps = 12;
```

5.4 Volume — ශ්‍රී ලංකා ක්‍රමය

timber_volume.dart

```
/// A running total over a stack, added the way the trade adds it.
///
/// Each log's adi and angal are counted in their own column and the carry is
/// only taken at the very end: every angal is summed, then divided by 12,
/// and the whole part is carried into the adi column.
///
/// This is deliberately integer arithmetic. Summing each log's decimal cubic
/// feet instead would accumulate binary rounding error across a long stack
/// and could land the printed total a hair either side of a boundary, which
/// on an invoice reads as the app being unable to add up.
@immutable
class StackVolumeTotal {
```

Decimal cubic feet එකතු කිරීමේදී binary rounding error එකතු වෙනවා. Invoice එකක මුළු එකතුව හරියන්නේ නැත්නම්, user එකාට ජේන්තේ app එකට එකතු කරන්න බැරි බවයි.

5.5 Password Policy

password_policy.dart

```
static String? validate(String? password) {
  final value = password ?? "";

  if (value.isEmpty) return "Enter a password";

  if (value.length < minLength) {
    return "Use at least $minLength characters";
  }

  if (!value.contains(RegExp(r'[A-Z]'))) {
    return "Add a capital letter";
  }

  if (!value.contains(RegExp(r'[a-z]'))) {
    return "Add a small letter";
  }

  if (!value.contains(RegExp(r'[0-9]'))) {
    return "Add a number";
  }

  // Anything that is not a letter, a digit or a space. Defining it by
  // exclusion rather than listing symbols means a password using a
  // character nobody thought to allow is not rejected for no reason.
  if (!value.contains(RegExp(r'^A-Za-z0-9\s')))) {
    return "Add a symbol, like ! or @";
  }

  return null;
}
```

Firebase එකෙන් enforce කරන්නේ අකුරු 6ක් විතරයි — වෙන කිසිම නීතියක් නෑ. ඒ නිසා අනෙක් හැම නීතියක්ම මේ validator එකේ තියෙන්න ඕන.

6. Screen Flows — කේතය සමඟ

6.1 Log Face Trace — handles 6

පරණ code එකේ ප්‍රශ්නය: pan එකක් වෙත හැම වතාවේම points 72න් ළඟම එක හොයාගෙන එක අදිනවා. ඇඟිල්ල ලොග් එක හරහා ගියොත් එක point එකක් අතාරලා තව එකක් අල්ලනවා — පිටිපස්සේ දිගට කුඩු පාරවල්.

log_face_trace_screen.dart

```
_Handle? _handleAt(Offset screenPoint, FittedImageMapper mapper) {
  _Handle? best;
  var bestDistance = _grabRadius;

  _handlePositions(mapper).forEach((handle, imagePoint) {
    final distance = (mapper.toScreen(imagePoint) - screenPoint).distance;

    if (distance <= bestDistance) {
      bestDistance = distance;
      best = handle;
    }
  })
}
```

ඇත්: grab radius එකක් තියෙනවා. ඇඟිල්ල handle එකකට ළඟ නැත්නම් null — ඒ කියන්නේ කිසිම දෙයක් වෙන්නේ නෑ.

_moveHandle()

```
void _moveHandle(_Handle handle, Offset imagePoint) {
  final shape = _shape!;

  if (handle == _Handle.majorCentre) {
    setState(() => _shape = shape.copyWith(centre: imagePoint));
    return;
  }

  final delta = imagePoint - shape.centre;

  final along = Offset(math.cos(shape.rotation), math.sin(shape.rotation));
  final across = Offset(-along.dy, along.dx);

  // A resize handle only ever changes the axis it belongs to; the reach
  // across the other axis is ignored. That is what makes a sloppy drag
  // still do the one comprehensible thing.
  switch (handle) {
    case _Handle.majorPlus:
    case _Handle.majorMinus:
      final reach = (delta.dx * along.dx + delta.dy * along.dy).abs();
      setState(() {
        _shape = shape.copyWith(semiMajor: math.max(reach, 6));
      });
    }
  }
```

Gesture handling — bug එකක් තිබුණු තැන

ඇයි tap callbacks නෑ?

```
// Pan callbacks only, deliberately. `onPanDown` already fires on
// every pointer down, tap or not, so adding tap callbacks alongside
// them does two kinds of damage: a plain tap runs `onDown` twice --
// marking a defect and immediately unmarking it -- and the moment a
// drag passes the tap slop the tap recogniser cancels, which used to
// let go of the handle the drag had just taken hold of.
return GestureDetector(
  // Keyed so a test can find the drawing surface and work out where
  // the handles are on screen.
  key: const ValueKey("trace-canvas"),
  onPanDown: (d) => onDown(d.localPosition),
  onPanUpdate: (d) => onMove(d.localPosition),
  onPanEnd: (_) => onUp(),
  onPanCancel: onUp,
```

6.2 Cutting — isolate එකක ගණනය කිරීම

optimal_cutting_screen.dart

```
setState(() => _planning = true);

// Off the UI thread: the search is long enough to freeze the screen, and
// a frozen screen is indistinguishable from a crashed one.
final comparison = await compute(SawingEngine.planBoth, request);

if (!mounted) return;
setState(() => _planning = false);

Navigator.push(
```

Search එකට desktop එකක තත්පර 0.5ක් යනවා. Phone එකක ඊට වඩා. UI thread එකේ කළොත් screen එක හිර වෙනවා — සහ හිර වුණු screen එකක් crash වුණු එකක් වගේම පේනවා.

6.3 Pattern එක ඡායාරූපය උඩ අඳින එක

sawing_patternPainter.dart

```
/// A point in plan millimetres, in widget coordinates.
Offset toScreen(Offset mm) => mapper.toScreen(
  Offset(
    faceOriginPx.dx + mm.dx / mmPerPixel,
    faceOriginPx.dy + mm.dy / mmPerPixel,
  ),
);
```

mm → image pixel → screen coordinate. `faceOriginPx` එකෙන් plan එක ඡායාරූපයේ නිවැරදි තැනට යනවා.

7. Diagnostics — ඇයි සහ කොහොමද

diagnostics_service.dart

```
Future<T> timed<T>(String module, Future<T> Function() action) async {
  final watch = Stopwatch()..start();

  try {
    final result = await action();
    watch.stop();

    await recordTiming(
      module: module, milliseconds: watch.elapsedMilliseconds);

    return result;
  } catch (error) {
```

හැම call site එකකටම stopwatch එකක් මතක තියාගන්න කියනවා වෙනුවට wrap කරනවා. මතක නැති වුණු timing එක තමයි හැම විටම වැදගත් වෙන එක.

p90 – mean නොවේ

```
Future<int?> percentile90(String module) async {
  try {
    final rows =
      await LocalDB.getDiagnostics(limit: LocalDB.diagnosticsLimit);

    final durations = <int>[
      for (final row in rows)
        if (row["kind"] == kindTiming &&
```

Requirement එකේ කියන්නේ 90th percentile එකෙන්. Mean එකෙන් හෙමින් runs හංගනවා — ඒවා තමයි target එක තියෙන්නේම අල්ලන්න.

7.1 Diagnostic store එක වැඩෙන්නේ නෑ

recordDiagnostic() – trim on write

```
await db.rawQuery(
  "DELETE FROM diagnostics WHERE id NOT IN "
  "(SELECT id FROM diagnostics ORDER BY id DESC LIMIT ?)",
  [diagnosticsLimit],
);
```

Rows 200යි. ලියන වෙලාවේම trim කරනවා — කියවන වෙලාවේ නොවේ. ඒ නිසා කවුරුවත් නොබලන phone එකක ගොනුව මාස ගණන් නිශ්ශබ්දව වැඩෙන්නේ නෑ.

8. Tests — කේතය නිවැරදි කියලා දන්නේ කොහොමද

Tests 327ක්. Test නම් වලින්ම architecture එක කියවෙනවා:

test/cloud_sync_test.dart

```
test('work survives the cloud being unreachable, and goes later', () async {
  final id = await LocalDB.createStack("Mahogany", 0);

  cloud.failure = Exception("no signal");

  await engine.queueStack(id);
  await engine.drain();

  // The old design pushed with unawaited() and no record: this stack
  // would simply never have existed in the cloud, and nobody would know.
  expect(cloud.stacks, isEmpty);
  expect(await LocalDB.pendingSyncCount(), 1);

  cloud.failure = null;
  await engine.drain();

  expect(cloud.stacks, hasLength(1));
  expect(await LocalDB.pendingSyncCount(), 0);
});
```

මේ test එකෙන් ඔප්පු කරන්නේ: cloud එක නැති වුණාම කිසිවක් නැති වෙන්නේ නෑ, සහ පස්සේ ඒක ඉබේම යනවා.

test/nfr_test.dart – NFR එකක් ඔප්පු කිරීම

```
test('the saved-records query stays fast with a full database', () async {
  // The requirement names 500 entries, so that is what is measured --
  // not 10, which any implementation passes.
  const entries = 500;

  for (var i = 0; i < entries; i++) {
    final stackId = await LocalDB.createStack(
      "Stack $i",
      0,
      customerName: "Customer ${i % 40}",
    );

    await LocalDB.addLogAndUpdateStackVolume(
      stackId: stackId,
      diameter: 20 + (i % 15),
      lengthFeet: 10,
      volume: 5,
    );
  }

  final watch = Stopwatch()..start();
  final stacks = await LocalDB.getStacks();
  watch.stop();

  expect(stacks, hasLength(entries));
  expect(
```

NFR එකක් ලේඛනයේ ලියනවා වෙනුවට මනිනවා. Entries 500ක් හදලා, retrieval එක තත්පර 2ට අඩුද කියලා බලනවා.

9. Database — සම්පූර්ණ Schema සාරාංශය

Table	PK	FK	Columns
users	uid	—	uid, name, email, phone, company, createdAt
stacks	id	userId → users	id, name, totalVolume, totalCost, createdAt, customerName, remarks, cloudId, userId, rate
logs	id	stackId → stacks	id, stackId, diameter, lengthFeet, volume, cost, createdAt, measurementSource, rawDiameterInches, deductionInches, diameterToleranceInches, measurementQuality, diameterProfile, cloudId, notes, smallEndDiameter, largeEndDiameter
defects	id	logId → logs	id, logId, kind, confidence, automatic, centreX, centreY, radius, imagePath, overlayPath, createdAt, cloudId, severity
cutting_patterns	id	logId → logs (UNIQUE)	id, logId, strategy, boardWidthMm, boardThicknessMm, bladeThicknessMm, boardCount, yieldPercent, boardVolumeCubicFeet, wasteVolumeCubicFeet, sawPasses, pricePerCubicFoot, createdAt, cloudId
reports	id	stackId / logId	id, stackId, logId, format, filePath, totalVolumeCubicFeet, totalCost, createdAt
diagnostics	id	—	id, kind, module, code, message, durationMs, createdAt
sync_outbox	id	—	id, entity, operation, localId, parentLocalId, cloudId, parentCloudId, queuedAt, attempts, lastError

9.1 සම්බන්ධතා (Relationships)



10. සාරාංශය — වැදගත්ම කරුණු 8

#	කරුණ	ඇයි වැදගත්?
1	Phone එක source of truth, cloud එක mirror	දැව ගබඩාවක signal නෑ
2	sync_outbox — offline queue	Fail වුණු upload එකක් නැති වෙන්නේ නෑ
3	cloudId හැම row එකකම	Restore කරාම duplicate වෙන්නේ නෑ
4	Provenance columns 6	Volume එකක් defend කරන්න පුළුවන්
5	cutting_patterns.logId UNIQUE	Schema එකෙන් enforce, code එකෙන් නොවේ
6	CloudTransport interface	Device නැතුව test කරන්න පුළුවන්
7	Volume එකෙන් score කරනවා	Board ගණන් කරොත් පටු board වලට ක්‍රියා
8	Tests 327	'වැඩ කරනවා' කියනවා වෙනුවට ඔප්පු කරනවා